

SoundHound CIE Interview Prep

Behavioral Interview Prep

Cognitive Implementation Engineer I — Amelia Global Services, Bengaluru
June 2026

Behavioral Interview — SoundHound Cognitive Implementation Engineer

Target: Cognitive Implementation Engineer I — Amelia Global Services, Bengaluru **Interviewers:** Pruthvi Bangarshettar, Sushmitha Venkatesh (Vilas & Vishal optional) **Duration:** 45 minutes. Jun 23, 4:30 PM IST.
Format: In-person, Bengaluru office (Primeco Union City, ITPL Main Road)

Coding: Known SoundHound Questions (Quick Reference)

You already passed the coding screen. But just in case they throw a quick one:

Known SoundHound LeetCode (from CodeJeet/Taro/AmbitionBox)

Problem	Difficulty	Pattern	Solution
Check if Array Is Sorted and Rotated	Easy	Array	Check if there's ≤ 1 break in sorted order
Valid Anagram	Easy	Hash Map	Counter(s) == Counter(t)
Two Sum	Easy	Hash Map	Store complement, check if seen
Group Anagrams	Medium	Hash Map	Key = sorted string $\rightarrow$ defaultdict(list)
First Missing Positive	Hard	Hash Map	Use array indices as hash keys

Solved Solutions (Pattern + Code)

Check if Array Is Sorted and Rotated

```
def check(nums):
    # Pattern: Count "breaks" where next < prev
    # In a sorted array, there should be at most 1 break
    breaks = 0
    for i in range(len(nums)):
        if nums[i] > nums[(i + 1) % len(nums)]:
            breaks += 1
    return breaks <= 1

# [3,4,5,1,2]  $\rightarrow$  1 break (5 $\rightarrow$ 1)  $\rightarrow$  True
# [2,1,3,4]  $\rightarrow$  1 break (2 $\rightarrow$ 1)  $\rightarrow$  False (wait – actually 1 break, but not sorted)
# Wait, let me trace: 2>1=break, 1<3=ok, 3<4=ok, 4>2=break  $\rightarrow$  2 breaks  $\rightarrow$  False ✓
```

Valid Anagram

```
from collections import Counter

def is_anagram(s, t):
    # Pattern: Compare character frequency maps
    return Counter(s) == Counter(t)

# is_anagram("listen", "silent")  $\rightarrow$  True
# is_anagram("hello", "world")  $\rightarrow$  False
```

Two Sum

```
def two_sum(nums, target):
    # Pattern: Store complement in hash map
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i
    return []

# two_sum([2, 7, 11, 15], 9)  $\rightarrow$  [0, 1]
```

Group Anagrams

```
from collections import defaultdict

def group_anagrams(strs):
    # Pattern: Sorted string as hash key
    groups = defaultdict(list)
```

```

for s in strs:
    key = ''.join(sorted(s))
    groups[key].append(s)
return list(groups.values())

# group_anagrams(["eat","tea","tan","ate","nat","bat"])
# → [["eat","tea","ate"], ["tan","nat"], ["bat"]]

```

First Missing Positive

```

def first_missing_positive(nums):
    # Pattern: Use array indices as hash keys
    # Place each number at its correct index (nums[i] at position nums[i]-1)
    i = 0
    while i < len(nums):
        correct_pos = nums[i] - 1
        if 0 <= correct_pos < len(nums) and nums[i] != nums[correct_pos]:
            nums[i], nums[correct_pos] = nums[correct_pos], nums[i]
        else:
            i += 1

    for i in range(len(nums)):
        if nums[i] != i + 1:
            return i + 1
    return len(nums) + 1

# first_missing_positive([3,4,-1,1]) → 2
# first_missing_positive([7,8,9,11,12]) → 1

```

CIE-Level Questions (From AmbitionBox Interviews)

These are what Cognitive Implementation Engineer candidates actually get asked:

Logic building:

```

# Q: Given a list of integers, find the pair that sums to a target
# Answer: Hash map approach
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i
    return []

```

String manipulation:

```

# Q: Reverse words in a string
# Answer: Split → reverse → join
def reverse_words(s):
    return ' '.join(s.split()[::-1])
# "the sky is blue" → "blue is sky the"

```

Counting:

```

# Q: Count frequency of each character
# Answer: Counter
from collections import Counter
def char_count(s):
    return dict(Counter(s))
# "hello" → {'h': 1, 'e': 1, 'l': 2, 'o': 1}

```

Basic data structure:

```

# Q: Check if string has all unique characters
# Answer: Set comparison
def has_unique(s):
    return len(s) == len(set(s))

```

If they ask "explain your approach": Talk about the PATTERN (hash map, two pointers), not just the code. That's what they're testing — your thinking process.

Your Profile (Grounded Facts Only)

Fact	Detail
Degree	B.E. Electronics & Communication Engineering, Ramaiah Institute of Technology, 2022-2026 (Graduated)
Internship	AI Engineer Intern, Beaut Group (SuperOwl), Feb-Jun 2026, on-site Bengaluru. Solo.
Projects	ROAST (solo, 100+ users), SYNAPSE (solo), ACARE (team — institutionally funded), SuperOwl (solo internship)
Skills	Python, JavaScript, FastAPI, WebSockets, Redis, SQLite, Firestore, Docker, GitHub Actions, AWS, React, React Native
Certifications	ML Specialization (Coursera — Andrew Ng), GenAI with LLMs (Coursera — AWS)
Open source	Contributed to Hermes Agent, AutoResearchClaw
Team vs Solo	ACARE was a team project. ROAST, SYNAPSE, SuperOwl internship — all solo.

The Real Playbook for 45 Minutes

5 min	— Introductions, small talk
15 min	— "Tell us about yourself" + project deep-dive
10 min	— Role-specific: "How would you approach X?"
10 min	— Behavioral questions (teamwork, challenges, motivation)
5 min	— Your questions to them

They're assessing 4 things:

1. Can I work with you? (communication, energy)
2. Can you learn Amelia? (problem-solving, not "do you know it")
3. Do you understand conversational AI? (intents, flows, integrations)
4. Can you code basic logic? (if they ask coding)

Nobody expects you to know Amelia. The role is Level I (0-1 yr). They'll train you. They want: smart, curious, communicative, picks up new tech.

1. Tell Me About Yourself (2 Minutes)

What They're Listening For

- Can you tell a coherent story? → communication
- Do you pick relevant things? → judgment
- Do you connect to THIS role? → homework done
- Is it natural or rehearsed? → authenticity

Full Script (Practice Out Loud)

"I've just completed my B.E. in Electronics & Communication Engineering at Ramaiah Institute of Technology. I've been building AI and backend systems for the past year — shipping production code, deploying to real users. I built four systems that connect to this role. First, SuperOwl — my internship project. I independently built a multi-tenant voice AI platform where businesses automate customer calls. The system uses VAPI for telephony, has per-tenant AI behavior, dynamic prompts, and escalation logic. It connects to Slack, Firebase, FCM — multiple external integrations. That's the same pattern as implementing Amelia for different enterprise clients. Second, ROAST — a resume analysis platform with 6 AI agents running in parallel with fallback chains and circuit breakers across 5 LLM providers. It's used by 100+ real users. Third, ACARE — a voice-controlled surgical robot. This was a team project, institutionally funded. I built the voice pipeline and the agentic planner. The robot has a 6-layer deterministic safety kernel. Fourth, SYNAPSE — an 8-node LangGraph reasoning engine with RAGAS evaluation, connected to a knowledge graph with 10K+ entities. I'm excited about this role because it's configuring Amelia for enterprise clients — building integrations, designing conversation flows, iterating with clients. I've done exactly that pattern in SuperOwl. I want to bring

that to SoundHound."

What NOT to Say

- ❌ "I started in computer science" — you're ECE. Don't claim CS.
- ❌ "I coordinated with team members" — you were solo on 3 out of 4 projects.
- ❌ "We worked together to..." — say "I" for solo work, "the team" for ACARE.

Follow-Up: "Which project was hardest?"

"ACARE, because it's physical. When software breaks, you get a log. When a robot breaks, things move wrong. The safety kernel wasn't optional — it was mandatory. That taught me to design for failure, not just success."

2. Why SoundHound

What They Want

- You understand their TECHNICAL differentiators
- You know why they matter for enterprise clients
- You've built similar things

Full Script

"SoundHound stands apart because you don't just wrap an LLM in a microphone. Your Speech-to-Meaning architecture fundamentally bypasses the error cascade of traditional ASR→NLU pipelines. I've seen this cascade problem firsthand — Deepgram would mis-transcribe a medical term in ACARE, the intent parser would misinterpret, and the robot would do the wrong thing. A direct meaning path would have been significantly more robust.

I'm also drawn to the Agentic+ architecture specifically. The three-layer model — Entities, Cognitive Functions, and AI Agents — is elegant. Instead of writing rigid scripts, you define building blocks and let AI agents orchestrate them dynamically based on conversation state. That's exactly what I built in SuperOwl: defining tools (`notify_owner`, `endCall`) and letting the AI decide which to use based on the situation. The difference is Amelia does this at enterprise scale with MCP integrations, guardrails, and omnichannel support.

And the company trajectory is interesting — founded 2005, years of voice AI research, now deploying at scale through Amelia and OASYS. You've been iterating on voice AI longer than most companies have existed."

If They Ask: "What do you know about Amelia?"

"Amelia 7 is SoundHound's enterprise conversational AI platform built on the Agentic+ framework. The architecture has three layers: Entities (business nouns like Account, Order), Cognitive Functions (atomic actions like `checkBalance`, `resetPassword`), and AI Agents (dynamic orchestrators that listen, assess state, reason, and act). Instead of rigid scripts, you define building blocks and let AI agents orchestrate them based on conversation state. The platform supports omnichannel (voice, chat, web), connects to enterprise systems via MCP or REST, has guardrails for compliance, and includes an Agent Console where AI assists human reps during live calls."

3. Why This Role

Full Script

"This role is the intersection of three things I enjoy: conversational AI, client-facing problem solving, and building integrations.

First, conversational AI — I've built voice pipelines, intent parsers, and dialogue systems in ACARE and SuperOwl. I understand how they work and where they fail. Configuring Amelia's ontologies and BPNs is applying that understanding to a real enterprise platform.

Second, client-facing problem solving — every client has different systems, different data, different needs. The Transversal Analysis → Blueprint → Configure → Test → Refine loop is how I already work: understand the domain, design a solution, build it, iterate.

Third, integrations — I've connected VAPI with Firebase, Slack, FCM, and multiple LLM providers in SuperOwl. That's the same integration work this role does — connecting Amelia to enterprise backend systems."

4. Strengths (Honest, Grounded in Your Resume)

Strength 1: You Ship Alone

"I've independently built and deployed 3 production systems. ROAST has 100+ real users. SuperOwl was a multi-tenant platform I architected end-to-end. I don't need handholding — I take an idea to production on my own."

Why this matters for CIE: The role involves autonomous sprint work with client deliverables. Solo ownership is a feature, not a bug.

Strength 2: You Build for Failure

"Every system I've built has resilience built in. ROAST has circuit breakers and 5-level fallback chains. ACARE has a 6-layer deterministic safety kernel. SuperOwl has escalation logic when the AI can't handle a call. I design for when things fail, not just when they succeed."

Why this matters for CIE: Enterprise clients expect uptime. Amelia deployments must be robust.

Strength 3: You Understand Voice AI End-to-End

"Two voice AI systems — ACARE (VAD → ASR → intent → planning → TTS) and SuperOwl (VAPI integration, webhook handling, real-time transcripts, HITL). Different architectures, both shipped. Voice AI is the domain you work in."

5. Weaknesses (Honest, Grounded)

Weakness 1: Solo on Most Projects

"I've been mostly solo. ROAST, SYNAPSE, SuperOwl — I built them alone. ACARE was the only team project. I want to learn how to work in cross-functional agile teams — that's actually one reason I'm interested in this role. The CIE role involves working with Technical Architects, Cognitive Project Leads, and integration experts. I want to build that muscle."

Why this works: It's honest, it's self-aware, and it connects to why you want THIS role.

Weakness 2: No Java/Groovy Experience

"My primary languages are Python and JavaScript. The role requires Java or Groovy. I'm confident I can pick up Groovy quickly — the patterns transfer from Python. I've learned new frameworks fast before: I picked up LangGraph from zero to a working 8-node pipeline in a week."

Why this works: Acknowledges the gap. Shows you've learned fast before with a concrete example.

Weakness 3: Over-Engineer Early

"I tend to add complexity before I need it. In ROAST, I built the ingestion pipeline synchronously — it took 10 hours for 70 combinations. I could've started with async concurrency from day one, but I was over-thinking rate limits. A simple semaphore fixed it — under an hour. I've learned to start simple and add complexity when the data shows I need it."

6. Hardest Technical Challenge (STAR Format)

STAR Structure

S – Situation (2 sentences)
T – Task (1 sentence)
A – Action (70% of your answer – YOUR thinking, YOUR decisions)
R – Result (measurable outcome + what you learned)

Story 1: SuperOwl VAPI 7.5-Second Timeout

Situation: SuperOwl is a multi-tenant voice AI platform. When a customer calls, VAPI sends an HTTP webhook to my backend asking for the AI assistant configuration. The backend must respond within 7.5 seconds or the call fails.

Task: I needed to resolve which business the call is for (via 5-layer phone number lookup), load business settings, choose the right prompt mode (personal, business, or hybrid), configure features, and return the assistant config — all within 7.5 seconds.

Action: I designed a 5-layer ANI resolution system. Layer 1: parse the SIP Diversion header for the caller's original number. Layer 2: check SIP From/To headers. Layer 3: lookup in an in-memory phone-to-business map. Layer 4: query Firestore by business phone. Layer 5: query by owner's personal number. Each layer tries the next if the previous fails.

For the timeout constraint, I separated the critical path (return assistant config) from the non-critical path (Slack notification, FCM push). The response goes back to VAPI immediately. Notifications are sent asynchronously as fire-and-forget background tasks.

For prompt mode selection, I built a hybrid system that detects intent first — "is this about the business or personal?" — and routes to the appropriate template. Each template has variable substitution: business name, hours, services, greeting.

Result: Calls consistently connect within 2-3 seconds. The 5-layer resolution catches edge cases where standard SIP headers don't have the caller's number. Fire-and-forget notifications keep the response time under the 7.5s threshold.

I learned: Real-time systems have hard constraints. You can't retry if you miss the deadline. Separate critical from non-critical work. Always have a fallback path.

Story 2: ROAST Multi-Provider Fallback

Situation: ROAST runs 6 AI agents, each calling an LLM provider. Free-tier APIs have rate limits and outages. One provider failing blocks the entire pipeline.

Task: Make provider failures invisible to the user.

Action: Three-part solution:

1. **Circuit breaker** per provider — 3 consecutive failures → skip for 5 minutes
2. **Fallback chain** per agent — e.g., Review Agent tries llama-3.3-70b → gpt-oss-20b → qwen3-32b → gemini → NIM → OpenRouter
3. **Deterministic fallback** — if ALL LLMs fail, generate a valid review using Python code only

I also implemented distributed key rotation via Redis INCR for Groq's API keys — multiple keys, round-robin distribution.

Result: Zero pipeline failures from provider issues. The deterministic fallback has triggered twice in production — lower quality, but the user always gets something.

I learned: Resilience isn't about preventing failure. It's about making failure invisible.

Story 3: ACARE SPI Communication Bug (Team Project)

Situation: In ACARE, a voice-controlled surgical robot, I was building the SPI link between a Raspberry Pi 5 and a Teensy 4.1 microcontroller. The Pi5 sends target joint positions, the Teensy sends back actual positions — all over SPI at 10 MHz.

Task: The Teensy was receiving corrupted data — joint angles that didn't make physical sense, causing the robot to twitch.

Action: I spent 3 days systematically isolating the issue. I wrote a byte-echo test — Pi sends 0x55, Teensy should reply with 0x56. The echo was wrong. So it wasn't a software logic bug — it was a hardware communication issue.

I tested: wiring (re-seated), voltage levels (3.3V stable), clock speed (reduced to 1 MHz — still broken), SPI mode (tried all 4). Then I realized the Teensy 4.1's Arduino SPI library doesn't have proper slave mode. The library assumes it's always master.

I switched to direct register configuration — manually setting LPSPI4 control registers for slave mode, writing a single ISR on the chip-select pin for both edges. This bypassed the library entirely.

Result: Clean, reliable SPI at 10 MHz with 37-byte frames.

I learned: When a protocol doesn't work, move down one layer of abstraction. Don't keep tweaking software — check the physical layer and driver layer first.

7. What Would You Do Differently

"With ROAST's ingestion pipeline, I built it synchronously. Each market combination took 10 minutes. All 70 took 10+ hours.

I was worried about rate limits. So I thought sequential was safer. But watching it run for 10 hours was painful.

I refactored to use `asyncio.gather` with a semaphore of 3. Same work completed in under an hour. The rate limits never triggered — the semaphore handled it.

What I'd do differently: start concurrent from day one. The fear of rate limits was irrational — a simple semaphore solves it. I also would've added progress logging from the start so I could estimate completion time."

8. How Do You Stay Updated on AI

"I follow a few focused sources. Karpathy on Twitter for deep technical breakdowns. The Batch from DeepLearning.AI for staying current. LMSYS Arena to track which models actually perform well.

But I learn most by building. When a new model comes out, I test it against my project's prompts. I ran DeepSeek V3 against ACARE's planner prompts — every call timed out. So I benchmarked Nemotron-49B, Groq 70B, and GPT-OSS 120B. Documented success rates and latency. Switched to Nemotron as primary.

Theory tells you what should work. Building tells you what actually works."

9. Where Do You See Yourself in 3-4 Years

"In the near term — first year — I want to master the Amelia platform. Ship multiple enterprise deployments end-to-end. Understand the common client patterns, the integration challenges, the pitfalls. I want to be the person who can take a new client's requirements and map them to Amelia's capabilities independently.

By year 2-3, I want to grow into a solutions architect role — designing the architecture for complex enterprise deployments, not just executing them. I want to understand the full picture: client needs, technical constraints, platform capabilities, and tradeoffs.

This role is the foundation. You can't design enterprise AI systems without understanding how they're actually deployed and configured. That's what I want to learn here."

10. "Why Should We Hire You?"

"Because I've already built what this role requires — voice AI pipelines, multi-agent systems, enterprise integrations, and production deployments with real users. I've done it solo, with free-tier APIs, on limited hardware. The learning curve on Amelia will be short because the concepts — ontologies, BPNs, automatas, integrations — are things I've worked with in different forms. I can start contributing from week one."

11. Common Behavioral Questions

"Tell us about a time you worked with a difficult stakeholder"

"In my SuperOwl internship, the business founder wanted a feature that would have required changes to VAPI's platform — something outside our control. Instead of saying 'we can't,' I explained the constraint and offered an alternative: we could handle the logic on our end via webhooks, even if we couldn't change VAPI's behavior.

They accepted. The lesson: constraints aren't walls. Clients appreciate honesty with options."

"How do you handle disagreements?"

"I focus on data and tradeoffs. When I had a design choice in ROAST — whether to use Redis or in-memory storage — I benchmarked both. In-memory was faster per request, but Redis survived restarts and scaled across instances. The data made the decision. I find that when you present tradeoffs clearly, disagreements resolve naturally."

"Describe a time you had to learn something new quickly"

"For SYNAPSE, I needed LangGraph — a framework I'd never used — to build an 8-node reasoning pipeline. I spent 2 days on the tutorial, built a minimal 2-node graph to verify my understanding, then iterated from there. Within a week, I had the full pipeline working with conditional edges and state persistence. My approach: build the smallest possible thing first, confirm it works, then expand."

"Tell us about a time you made a mistake"

"In ROAST's early version, I built the ingestion pipeline synchronously. Each market combination took 10 minutes sequentially — 70 combinations meant 10+ hours. I thought sequential was safer because I was worried about hitting API rate limits."

The mistake was optimizing for the wrong risk. Rate limits were manageable with a simple semaphore. The real cost was time — 10 hours of waiting. When I refactored to `asyncio.gather` with a semaphore of 3, it finished in under an hour. Same work. No rate limit issues.

What I learned: Start concurrent from day one. The fear of rate limits was irrational — a simple semaphore solves it. I also started adding progress logging so I could estimate completion time."

"How do you handle changing requirements?"

"During my SuperOwl internship, the founder would often add features mid-sprint — a new Slack command here, a different notification format there. At first, I'd just build whatever was asked. But that led to a messy codebase and missed deadlines."

I started pushing back — not saying 'no,' but asking clarifying questions: 'Is this a must-have for launch or a nice-to-have?' and 'If we add this, what should we drop?' This helped the founder prioritize. We'd agree on what to defer to v2.

The core habit I learned: when requirements change, update the plan first — don't just start coding the new thing. Write down the change, estimate effort, negotiate scope."

"Explain a technical concept to a non-technical person"

Scenario: In the SuperOwl internship, the business founder didn't understand why some calls had a delay before the AI assistant responded.

My explanation: 'Imagine you're calling a business. The receptionist needs to check a binder to find who handles your type of request, then read a script before answering. That lookup takes time. Our system does the same thing — it needs to figure out who you're calling, pull up their settings, and build the right script. The delay is the lookup time.'

Why it worked: Instead of talking about VAPI webhooks, ANI resolution, or 7.5-second timeouts, I used an analogy the founder already understood — how a real receptionist works. The latency wasn't a bug; it was research time."

"Tell us about a time you took initiative beyond your role"

"During my SuperOwl internship, my main deliverable was the voice AI platform. But I noticed the team didn't have a testing strategy — every deployment was manual, and regressions were common."

No one asked me to fix this. But I spent evenings building a test suite: unit tests for the storage layer, integration tests for the VAPI webhook flow, and a local Firestore emulator setup. I also added GitHub Actions for CI.

The impact: deployment confidence went up. We caught two regressions before they hit production in the first week alone. The founder noticed and started requiring tests for every new endpoint.

The lesson: Initiative isn't about doing extra work. It's about seeing a problem and fixing it without waiting for permission."

"Why ECE and not Computer Science?"

"I chose ECE because I was interested in AI and building intelligent systems. In my first year, I started exploring machine learning on my own — Andrew Ng's course, building small models, understanding how neural networks work. By second year, I was building production AI systems."

The degree label didn't matter. What mattered was that I was building — voice AI pipelines, multi-agent systems, production backends. ECE gave me the engineering foundation; I built the AI expertise myself through

projects and deployment experience.

Actually, my ECE background helped in unexpected ways — signal processing concepts transferred directly to understanding voice AI pipelines. The core skills I use daily (Python, APIs, system design) are things I learned building projects, not in a classroom."

"What do you know about our competitors?"

"The main players in enterprise conversational AI are Amelia, Kore.ai, Nuance (Microsoft), Google Contact Center AI, and AWS Connect.

What sets SoundHound apart is **Speech-to-Meaning** — instead of the traditional ASR → NLU pipeline where errors cascade from one stage to the next, Amelia goes directly from audio to meaning. This is fundamentally different from how Google or AWS approach it.

The **Agentic+ framework** is also unique — most competitors still rely on rigid dialogue trees or basic LLM wrappers. Amelia's three-layer architecture (Entities → Cognitive Functions → AI Agents) with dynamic orchestration is ahead of what Kore.ai or Nuance offer.

And **OASYS** is new — being model-agnostic means enterprises aren't locked into one LLM. Competitors typically force you into their ecosystem."

12. Your 3 Questions to Ask

Q1 (role-specific):

"In your experience deploying Amelia for enterprise clients, how do you decide when a use case needs a BPN vs when the LLM-powered Agentic+ flow can handle it?"

Q2 (learning):

"What does the onboarding process look like for a Cognitive Implementation Engineer new to Amelia? How long before they're working on client deployments?"

Q3 (curiosity):

"What's a client deployment you're particularly proud of — where Amelia solved something that traditional IVR or basic chatbots couldn't?"

Don't ask: What does this team do? (you should know), Salary questions (HR handles separately), When will I hear back? (they'll tell you)

Quick Reference Card

YOU → 2 min intro: ECE at RIT → 4 projects (SuperOwl, ROAST, ACARE team, SYNAPSE) → CIE role

STAR → Situation (2 sentences) → Task (1 sentence) → ACTION (70%) → Result + lesson

SOUNDHOUND → Speech-to-Meaning + Amelia Agentic+ + 2005 founding + 26 countries

THIS ROLE → Configure Amelia for enterprises. BPNs, ontologies, automatas, integrations.

YOUR FIT → SuperOwl = same pattern. Voice AI + multi-agent + integrations.

STRENGTHS → Ship alone, build for failure, voice AI end-to-end

WEAKNESSES → Solo (need team experience), no Java/Groovy (learn fast), over-engineer early

YOUR PROJECTS:

SuperOwl (solo) — Multi-tenant voice AI platform, VAPI integration, HITL

ROAST (solo) — 6 agents, 5 LLM fallbacks, 100+ users, circuit breakers

ACARE (team) — Voice pipeline, agentic planner, 6-layer safety kernel

SYNAPSE (solo) — 8-node LangGraph, 10K+ entity knowledge graph, RAGAS